AeroAndes Reservas: diseño de comunicación entre procesos

Repositorio del equipo: https://github.com/iMouraad/APPD_MPZ

Objetivo del documento

Explicar cómo el diseño de AeroAndes combina comunicación síncrona y asíncrona para evitar la sobreventa de asientos, procesar pagos que pueden tardar hasta 30 segundos, enviar notificaciones y mantener el embarque durante una desconexión. Los sistemas de reserva aérea dependen de componentes de cómputo, redes y terminales que deben responder a transacciones en línea [1]; por eso el diagrama separa claramente los canales de venta, el centro de datos, los sistemas externos y la red local del aeropuerto.

1. Vista general del sistema

El pasajero puede usar la aplicación web o móvil y una agencia de viajes puede usar su propio sistema. Ambos canales llegan al **Servicio de reservas**. Este servicio coordina inventario, pagos y el estado de la reserva; no administra por sí solo la disponibilidad de los asientos.

La arquitectura separa responsabilidades porque una falla parcial no debería detener todo el sistema. La confiabilidad de una aplicación distribuida depende tanto de sus componentes como de sus mecanismos de detección y recuperación de fallos [2].

2. Búsqueda, retención y confirmación del asiento

1. En **I1** e **I2**, el cliente o la agencia llama a Reservas mediante REST sobre HTTPS. La comunicación es síncrona porque necesita una respuesta para continuar.
2. En **I3**, Reservas consulta, retiene o confirma un asiento mediante gRPC sobre HTTP/2. Inventario es la autoridad sobre la disponibilidad.
3. En **I4**, Inventario actualiza la base de datos mediante SQL/TCP. La retención dura 10 minutos y la confirmación debe comprobar que la retención siga vigente y pertenezca a la misma reserva.
4. En **I5**, Reservas guarda el estado de la operación. Así puede responder una consulta posterior aunque el cliente haya perdido la conexión.

Idea clave: dos solicitudes pueden competir por el mismo asiento, pero solo una transacción debe cambiarlo de retenido a confirmado. La consistencia fuerte se concentra en Inventario y en la base de datos; no se intenta resolver la sobreventa con mensajes asíncronos.

3. Pago sin bloquear al cliente durante 30 segundos

En **I6**, Reservas llama a Pagos mediante gRPC y recibe en hasta 2 segundos el estado inicial del intento. Pagos registra una clave idempotente y después llama a la pasarela por HTTPS en **I7**. La pasarela puede tardar hasta 30 segundos, por lo que este trabajo continúa fuera de la solicitud original del cliente.

Cuando existe un resultado, Pagos publica `pago.aprobado` o `pago.rechazado` en RabbitMQ mediante **I8**. Reservas consume el evento en **I9**. Si el pago fue aprobado, solicita a Inventario la confirmación del asiento. El evento puede llegar más de una vez; por eso Reservas debe reconocer el identificador del intento y procesarlo de forma idempotente.

Idea clave: "síncrono" no significa que todo el pago termine dentro de la petición del usuario. La respuesta rápida comunica que el intento fue registrado; el resultado final llega por mensajería.

4. Notificaciones y pase de abordar

Después de confirmar la reserva, Reservas publica `reserva.confirmada` y `pase.emitido` en **I10**. RabbitMQ entrega el trabajo a Notificaciones en **I11**; el servicio envía correo o SMS en segundo plano por **I12**, con una llamada síncrona al proveedor. La cola debe ser durable para conservar el mensaje si Notificaciones está temporalmente caído.

El mismo pase llega al Servidor de embarque mediante **I13**. Así se actualiza una copia local del manifiesto antes de que el pasajero llegue a la puerta.

5. Operación del aeropuerto sin conexión al centro de datos

Los lectores de puerta y los quioscos se comunican con el Servidor de embarque mediante sockets TCP en la red local (**I15** y **I16**). Para los 40 lectores simultáneos de I15, la respuesta es síncrona y debe tardar menos de un segundo. Si se corta el enlace con el centro de datos, el aeropuerto valida los pases ya presentes en su manifiesto local y registra los usos pendientes.

Cuando vuelve la conexión, Embarque publica `embarque.registrado` por **I14**. Los pases nuevos que no llegaron antes del corte necesitan revisión manual; el diseño no puede aceptar con seguridad datos que nunca recibió.

Tabla de interacciones y justificación

ID	Interacción y mensaje	Tipo, protocolo y middleware	Justificación
I1	Cliente → Reservas. Buscar vuelos, retener, pagar y consultar estado.	REST sobre HTTPS, síncrona.	**R1, R4.** El cliente necesita ver la disponibilidad al buscar. Al iniciar el pago recibe el estado del intento y puede consultarlo mientras la pasarela procesa el cobro.
I2	Agencia → Reservas. Solicitar una reserva.	REST sobre HTTPS, síncrona.	**R2, R3.** La agencia usa el mismo servicio de Reservas que el cliente. Ambos canales pasan por Inventario para retener y confirmar asientos.
I3	Reservas → Inventario. Consultar, retener y confirmar asiento.	RPC síncrona con gRPC sobre HTTP/2.	**R1–R3.** Reservas espera la decisión de Inventario antes de informar que un asiento quedó retenido o confirmado. El objetivo de 1.200 consultas por segundo y p95 menor a 300 ms [3] debe comprobarse desde la solicitud del cliente hasta la respuesta.
I4	Inventario → BD. Consultar disponibilidad y actualizar el estado del asiento.	SQL sobre TCP, síncrona, con controlador PostgreSQL.	**R2, R3.** La retención vence a los 10 minutos. Al confirmar, una transacción breve comprueba que el asiento siga retenido por esa reserva y que la retención esté vigente. Si compiten dos solicitudes, solo una puede ocuparlo [4, sec. 13.3.2].
I5	Reservas → BD. Guardar la reserva y su estado.	SQL sobre TCP, síncrona, con controlador PostgreSQL.	**R3, R4.** Guardar el estado permite responder cuando el cliente consulta qué pasó con su compra. Reservas registra la confirmación después de que Inventario acepta el cambio del asiento.

ID	Interacción y mensaje	Tipo, protocolo y middleware	Justificación
I6	Reservas → Pagos. `IniciarPago` con reserva, monto y clave.	RPC síncrona con gRPC sobre HTTP/2 [5]. Objetivo del diseño de hasta 2 s para registrar el intento.	**R4.** Pagos devuelve el estado inicial del intento una vez registrado. La llamada puede terminar mientras el cobro sigue en curso.
I7	Pagos → Pasarela. Cobrar con una clave idempotente.	REST sobre HTTPS, síncrona desde Pagos. Tiempo límite de 30 s.	**R4, F1.** Pagos espera la respuesta de la pasarela fuera de la solicitud del cliente. Si necesita reintentar, reutiliza la clave del mismo intento según el contrato de la pasarela.
I8	Pagos → RabbitMQ. Publicar `pago.aprobado` o `pago.rechazado`.	Mensajería asíncrona con AMQP y RabbitMQ.	**R4.** El resultado se publica cuando termina el cobro. La solicitud inicial del cliente puede haber terminado para entonces.
I9	RabbitMQ → Reservas. Entregar el resultado del pago.	Mensajería asíncrona con AMQP y RabbitMQ.	**R3, R4.** Al recibir un pago aprobado, Reservas solicita la confirmación del asiento por I3. Si RabbitMQ vuelve a entregar el evento, Reservas reconoce el intento y evita procesarlo dos veces [6].
I10	Reservas → RabbitMQ. Publicar `reserva.confirmada` y `pase.emitido`.	Mensajería asíncrona con AMQP y RabbitMQ.	**R3, R5.** Los eventos salen después de guardar la confirmación. A partir de ellos pueden avanzar el aviso al pasajero y la actualización del manifiesto.
I11	RabbitMQ → Notificaciones. Entregar `reserva.confirmada` y `pase.emitido`.	Mensajería asíncrona con AMQP y RabbitMQ.	**R5, F2.** Una cola durable conserva el trabajo pendiente si Notificaciones se cae. El objetivo se mide hasta que el aviso llega al pasajero, en menos de un minuto después del pago.
I12	Notificaciones → Proveedor de correo y SMS. Enviar el aviso y el pase.	SMTP o HTTPS. Llamada síncrona al proveedor desde el proceso de envío en segundo plano.	**R5.** El envío ocurre después de responder al cliente. Notificaciones puede esperar la respuesta del proveedor en su propio proceso y reintentar si falla.
I13	RabbitMQ → Embarque. Entregar `pase.emitido` para actualizar el manifiesto.	Mensajería asíncrona con AMQP y RabbitMQ.	**R6, F3.** El servidor guarda los pases autorizados en el aeropuerto antes de que los necesiten los lectores.
I14	Embarque → RabbitMQ. Publicar `embarque.registrado`.	Mensajería asíncrona con AMQP y RabbitMQ. Almacenamiento local y envío posterior.	**R6, F3.** El servidor registra el uso del pase en el aeropuerto y lo comunica al centro de datos cuando hay conexión. La respuesta al lector puede salir antes.
I15	Lectores de puerta → Embarque. `RES|VUELO|ASIENTO` → válido o inválido.	Sockets TCP, síncrona en la red local [7, sec. 2.2].	**R6.** La puerta necesita una respuesta para decidir si permite el paso. El mensaje tiene formato fijo y se valida localmente. La meta es responder en menos de un segundo con 40 lectores conectados.
I16	Quioscos → Embarque. Validar pase o hacer *check-in*.	Sockets TCP, síncrona en la red local.	**R6, F3.** El quiosco también necesita un resultado inmediato. Durante un corte puede validar pases que ya figuren en el manifiesto local.

Alternativas evaluadas

REST/HTTPS en I1–I2 y gRPC en I3 e I6 (R1–R4). Los canales externos usan una API HTTPS que pueden llamar desde el navegador, la aplicación o el sistema de una agencia. Para las llamadas internas de Inventario y Pagos también se consideró REST con JSON. Se prefirió gRPC porque el contrato de servicios y mensajes se define en Protobuf y permite generar clientes a partir de ese contrato [8]. A cambio, ambos servicios deben mantener compatibles sus esquemas. Una cola para I3 obligaría a recibir por separado la decisión de Inventario, cuando el cliente necesita saber si el asiento quedó retenido en esa misma operación. El objetivo de 1.200 consultas/s y p95 menor a 300 ms requiere una prueba de carga.

Transacción SQL frente a confirmación diferida en I4 (R2, R3). Un modelo que publique la confirmación y deje que cada servicio actualice su copia después abre un intervalo en el que dos solicitudes pueden ver el mismo asiento como disponible. El diseño concentra la decisión en Inventario y comprueba la vigencia y el titular dentro de una transacción breve [4, sec. 13.3.2]. Si Inventario no puede confirmar el cambio, Reservas mantiene la compra pendiente. Los 10 minutos son el plazo de la retención registrada en la BD.

RabbitMQ frente a una llamada HTTP directa en I8–I14 (R4, R5, F2). Mantener abierta la petición original hasta que responda la pasarela trasladaría al cliente una espera de hasta 30 s. Un aviso HTTP directo de Reservas a Notificaciones también obligaría a guardar y reintentar la solicitud si ese servicio está caído. El broker permite entregar el resultado del pago y separar el envío de avisos del trámite de reserva. Para recuperar eventos se requieren colas durables, mensajes persistentes, confirmación del publicador y acuse del consumidor después de procesar. Una nueva entrega puede repetir un evento, por lo que el consumidor comprueba su identificador [6], [9]. El tiempo en cola cuenta para la meta de un minuto de R5.

TCP frente a UDP en I15–I16 (R6, F3). TCP entrega un flujo de bytes en orden y permite mantener abiertas las conexiones de los lectores [7, sec. 2.2]. Con UDP, la aplicación tendría que gestionar la pérdida de la solicitud o la respuesta, los reintentos y la identificación de duplicados. Con TCP, cada lector aún debe delimitar los mensajes, aplicar un tiempo límite y decidir qué hacer si pierde la conexión. Validar con el manifiesto local evita depender de la red externa durante el corte. El objetivo de menos de un segundo debe medirse con los 40 lectores simultáneos.

Respuestas a las situaciones de fallo

F1. La pasarela de pagos no responde

El cliente ve **«Pago en verificación»**. Si se agotan los 30 segundos de I7, todavía puede haber un cobro. Quizá la pasarela lo procesó y se perdió la respuesta. Pagos consulta el mismo intento cuando el proveedor lo permita.

Si pasan los 10 minutos sin que se confirme la reserva, el asiento se libera. Si después se comprueba que sí hubo un cobro, se tramita el reembolso y se informa al cliente del estado.

F2. Notificaciones está caído cuando se confirma la reserva

La reserva queda confirmada en la base de datos y el aviso queda pendiente. Para cubrir una caída entre guardar la confirmación en I5 y publicar en I10, proponemos guardar el evento pendiente en la misma transacción que registra la reserva confirmada.

Una vez publicado, RabbitMQ lo conserva en una cola durable hasta que Notificaciones lo procese. Si la caída se prolonga, el aviso puede superar el minuto.

F3. Se corta la conexión entre el aeropuerto y el centro de datos

Los lectores y quioscos siguen consultando al servidor de Embarque por la red local. El servidor valida con el manifiesto que recibió por I13 y registra cada uso localmente para evitar que el mismo pase se acepte dos veces.

Los pases nuevos y los cambios que aún no llegaron requieren revisión manual. Cuando vuelve la conexión, Embarque publica los usos pendientes por I14 y recibe las actualizaciones del manifiesto.

Criterios para comprobar la recuperación

Después de cada fallo no basta con que el servicio vuelva a responder. El equipo debe verificar que no existan cobros duplicados, que ningún asiento liberado aparezca confirmado, que los eventos pendientes hayan sido consumidos y que el manifiesto local coincida nuevamente con el estado central. Estas comprobaciones convierten la recuperación en un resultado observable y no solo en el reinicio de un proceso.

Declaración de uso de inteligencia artificial

Se utilizó ChatGPT (OpenAI) para organizar y revisar las notas. Se utilizó Codex (OpenAI) para preparar la tabla de interacciones y las respuestas a los fallos, verificar los enlaces de las fuentes, cotejar el texto con el diagrama e integrar los archivos en LaTeX.

Referencias

[1] «Airline Reservations Systems,» en *Control Systems Functions and Programming Approaches: Applications*, ép. Mathematics in Science and Engineering, D. N. Chorafas, ed., vol. 27.2. Academic Press, 1966, págs. 159-176, doi: 10.1016/S0076-5392(09)60268-X.

[2] W. Ahmed e Y. W. Wu, «A survey on reliability in distributed systems,» *Journal of Computer and System Sciences*, vol. 79, n.º 8, págs. 1243-1255, dic. de 2013, doi: 10.1016/j.jcss.2013.02.006.

[3] G. C. Guerrero Ulloa, «Rúbrica GA-TL-01-FORM,» Material docente, UTEQ, periodo 2026–2027, Anexo A, escenario 1, pp. 11–12. Archivo facilitado por el docente: Detalles..pdf. dirección: https://github.com/iMouraad/APPD_MPZ/blob/main/docs/Detalles..pdf

[4] PostgreSQL Global Development Group. «Explicit Locking.» Documentación oficial, sección 13.3; bloqueos de fila en la sección 13.3.2, visitado 10 de nov. de 2026. dirección: https://www.postgresql.org/docs/current/explicit-locking.html

[5] gRPC Authors. «Deadlines.» Documentación oficial, visitado 10 de nov. de 2026. dirección: https://grpc.io/docs/guides/deadlines/

[6] RabbitMQ. «Consumer Acknowledgements and Publisher Confirms.» Documentación oficial, visitado 10 de nov. de 2026. dirección: https://www.rabbitmq.com/docs/confirms

[7] W. Eddy, «Transmission Control Protocol (TCP),» RFC Editor, RFC 9293, ago. de 2022, W. Eddy, editor. Internet Standard, STD 7. Flujo de bytes en la sección 2.2, doi: 10.17487/RFC9293. visitado 10 de nov. de 2026. dirección: https://www.rfc-editor.org/rfc/rfc9293.html

[8] gRPC Authors. «Core concepts, architecture and lifecycle,» visitado 11 de nov. de 2026. dirección: https://grpc.io/docs/what-is-grpc/core-concepts/

[9] RabbitMQ. «Reliability Guide.» Documentación oficial, visitado 10 de nov. de 2026. dirección: https://www.rabbitmq.com/docs/reliability